# RutaSmart — Arquitectura y Diseño (SDD)

## IEEE 1016-2009

Sarah Isabella Peña Cortes

Docente: Ing. Jairo Armando Salcedo Aranda

Universidad de San Buenaventura, Bogotá · Optativa II: Desarrollo de Aplicaciones para Dispositivos Móviles · Noviembre de 2026

# Tabla de contenido

Información del documento ..... 3

1. Introducción ..... 4

2. Vista general de la arquitectura ..... 5

    2.1 Decisiones de diseño ..... 6

3. Tecnologías ..... 7

4. Diseño de clases (programación orientada a objetos) ..... 8

5. Estructuras de datos, arreglos y ciclos ..... 9

6. Algoritmos ..... 10

    6.1 Generación de alternativas (Planificador) ..... 10

    6.2 Motor de puntaje ..... 12

    6.3 Ejemplo real calculado por la app ..... 12

7. Persistencia ..... 13

8. Secuencia principal ..... 14

9. Navegación ..... 15

10. Manejo de excepciones ..... 16

11. Seguridad contra valores nulos ..... 17

12. Depuración ..... 18

13. Implementación multiplataforma ..... 19

14. Integración de inteligencia artificial ..... 20

15. Estructura del código ..... 23

## Información del documento

| Campo | Valor |
|---|---|
| Proyecto | RutaSmart — Asistente inteligente para selección de alternativas de transporte |
| Documento | RS-SDD-001 · Documento de Arquitectura y Diseño de Software |
| Estándar de referencia | IEEE 1016-2009 (Software Design Descriptions) |
| Versión | 1.0 |
| Autora | Sarah Isabella Peña Cortes (sipenac@academia.usbbog.edu.co) |
| Asignatura | Optativa II: Desarrollo de Aplicaciones para Dispositivos Móviles |
| Docente | Ing. Jairo Armando Salcedo Aranda |
| Institución | Universidad de San Buenaventura, Bogotá |
| Fecha | Noviembre de 2026 |

## 1. Introducción

Este documento describe **cómo está construido RutaSmart**: la arquitectura, el modelo de clases, la persistencia, los algoritmos, el manejo de excepciones, la seguridad frente a nulos, la depuración, la proyección multiplataforma y el asistente de IA.

Cada decisión del documento de diseño inicial (formato IEEE) se relaciona con el archivo de código que la implementa.

## 2. Vista general de la arquitectura

La aplicación sigue el patrón **MVVM (Model-View-ViewModel)**, recomendado por Google, con una separación por capas inspirada en Clean Architecture:

- **Presentación (ui/):** pantallas en Jetpack Compose y ViewModels que exponen el estado con StateFlow.
- **Dominio (domain/):** modelo, motor y casos de uso en Kotlin puro, sin dependencias de Android.
- **Datos (data/):** repositorios que abstraen Room, Firebase, Gemini y la red de transporte (patrón Repository).

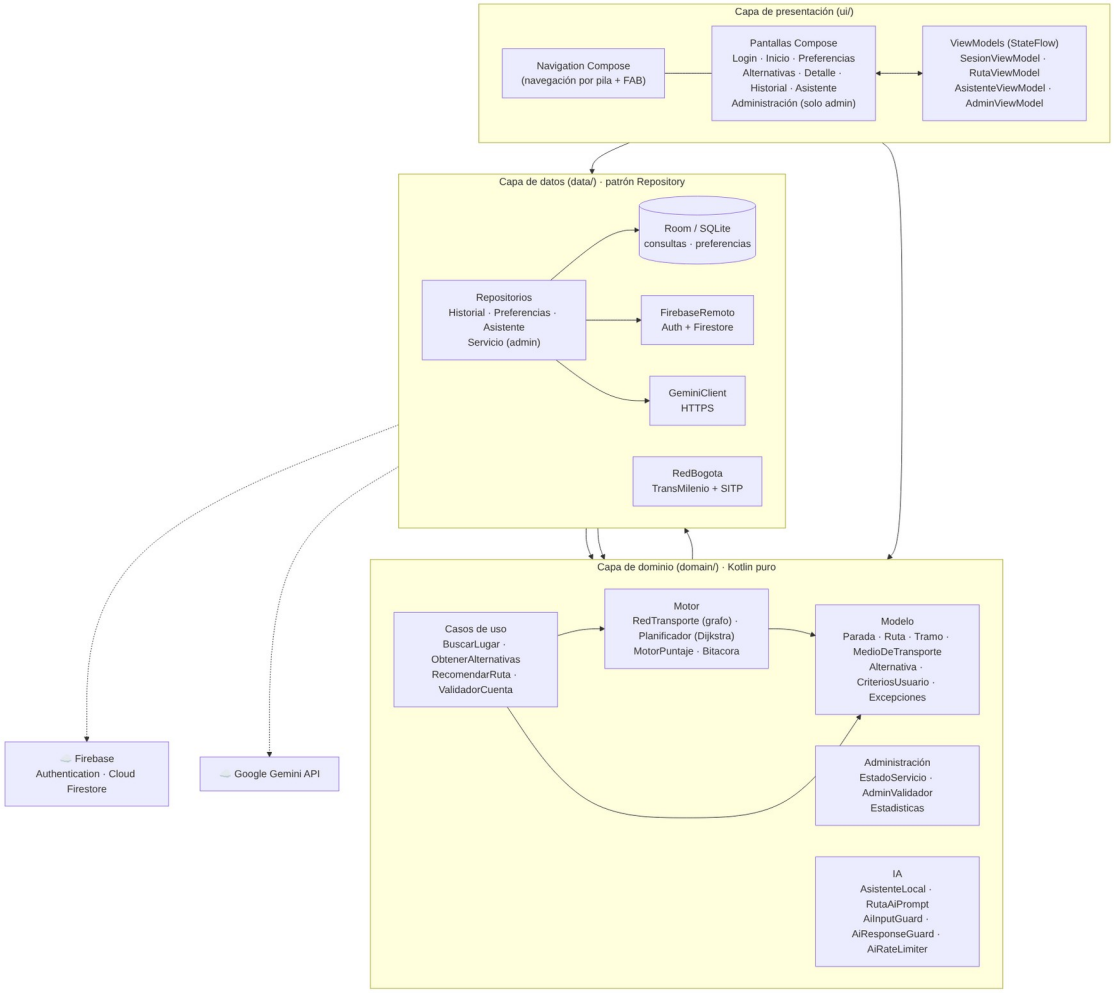



*Figura 1. Arquitectura por capas de RutaSmart*

**Flujo unidireccional:**

1. La vista envía eventos al ViewModel, por ejemplo buscar().

2. El ViewModel invoca los casos de uso, que usan el motor y los repositorios.
3. El resultado vuelve como un nuevo estado (StateFlow) que la vista observa y vuelve a dibujar.

## 2.1 Decisiones de diseño

| Decisión | Alternativa descartada | Motivo |
|---|---|---|
| Jetpack Compose + Material 3 | XML + Views | UI declarativa, menos código, estado observable. |
| MVVM + Repository | Lógica en las Activity | Separación de responsabilidades y dominio probable con JUnit. |
| Dominio en Kotlin puro | Dominio con dependencias Android | Pruebas rápidas y reutilización con Kotlin Multiplatform. |
| Motor de rutas propio (Dijkstra) + red de ejemplo | API de rutas de Google (de pago) | Sin costo, funciona sin internet, algoritmo explicable y probado. |
| Mapa esquemático en Canvas | Maps SDK (requiere clave y facturación) | Dibuja la ruta real sin dependencias; Google Maps se abre por Intent. |
| Room + Firestore | Solo nube | Funciona sin conexión y sincroniza cuando hay red. |
| IA híbrida (motor local + Gemini) | Solo LLM | La IA no inventa cifras y la app nunca se queda sin respuesta. |
| Inyección manual (Contenedor) | Hilt/Dagger | Menos configuración para el tamaño del proyecto. |

## 3. Tecnologías

| Área | Tecnología | Uso en el proyecto |
|---|---|---|
| Lenguaje / IDE | Kotlin 1.9.24 · Android Studio | Todo el código; depurador, emulador, Logcat, Layout Inspector. |
| Interfaz | Jetpack Compose (BOM 2024.06) · Material 3 | 8 pantallas, tema claro/oscuro, componentes accesibles. |
| Navegación | Navigation Compose 2.7.7 | Navegación por pila; argumento entero en el detalle. |
| Concurrencia | Kotlin Coroutines + Flow/StateFlow | Cálculo en Dispatchers.Default; red en Dispatchers.IO. |
| Persistencia local | Room 2.6.1 (KSP) | Tablas consultas y preferencias; consultas con Flow. |
| Nube | Firebase Auth + Cloud Firestore (BoM 33.1.2) | Cuentas, historial y preferencias por usuario; rol de administrador y estado del servicio. |
| IA | Google Gemini (REST generateContent) | Asistente «Rutina» con HttpURLConnection y org.json. |
| Pruebas | JUnit 4 | 80 pruebas unitarias del dominio. |
| CI/CD | Azure Pipelines | Lint, pruebas unitarias y APK de depuración. |

## 4. Diseño de clases (programación orientada a objetos)

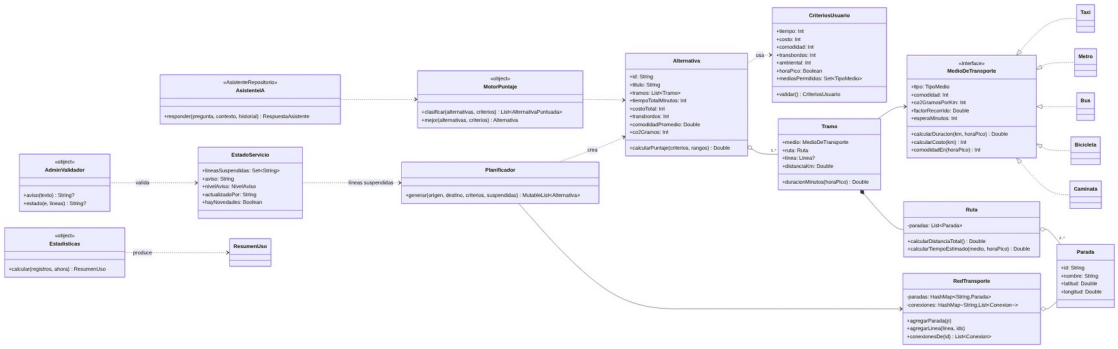



*Figura 2. Diagrama de clases del dominio*

| Concepto de POO | Aplicación en RutaSmart | Archivo |
|---|---|---|
| Clases y objetos | Parada, Lugar, Linea, Ruta, Tramo, Alternativa, CriteriosUsuario, RedTransporte, Planificador. | domain/model, domain/motor |
| Encapsulamiento | Atributos val (solo lectura); Ruta guarda su lista privada y expone una copia; RedTransporte oculta sus HashMap. | Ruta.kt, RedTransporte.kt |
| Interfaz y polimorfismo | MedioDeTransporte define calcularCosto() y calcularDuracion(); Bus, Metro, Taxi, Bicicleta y Caminata la implementan. El motor los usa sin conocer la clase concreta. | MedioDeTransporte.kt |
| Agregación | Alternativa agrega una lista de Tramo; Ruta agrega paradas; RedTransporte agrega paradas y líneas. | Alternativa.kt |
| Composición | Tramo se compone de una Ruta; Alternativa calcula su puntaje usando CriteriosUsuario. | Ruta.kt, Alternativa.kt |
| Herencia | Solo donde hay relación «es un»: los medios implementan la interfaz; las excepciones heredan de RutaSmartException (sealed). | Excepciones.kt |
| Objetos singleton | Medios (fábrica), MotorPuntaje, Tarifas, Bitacora, RedBogota. | varios |

## 5. Estructuras de datos, arreglos y ciclos

| Elemento del diseño | Implementación real | Archivo |
|---|---|---|
| List<Parada> de una ruta | Ruta(paradas: List<Parada>) con las paradas en orden. | Ruta.kt |
| Array<MedioDeTransporte> | Medios.todos(): Array<MedioDeTransporte> con los 5 medios. | MedioDeTransporte.kt |
| HashMap<String, Parada> como grafo | RedTransporte: HashMap de paradas + HashMap de listas de adyacencia (Conexion). | RedTransporte.kt |
| MutableList<Alternativa> | Planificador.generar() devuelve MutableList<Alternativa>. | Planificador.kt |
| Cola de prioridad | PriorityQueue de Dijkstra ordenada por minutos acumulados. | Planificador.kt |
| Array de paradas de la red | RedBogota.PARADAS: Array<Parada> con 50 paradas. | RedBogota.kt |

**Ciclos de control:**

| Ciclo | Dónde | Para qué |
|---|---|---|
| for | Ruta.calcularDistanciaTotal() | Sumar la distancia entre paradas consecutivas. |
| for | RedTransporte.agregarLinea() | Crear conexiones entre paradas consecutivas de una línea. |
| for | Planificador (ORIGEN → paradas) | Recorrer la red buscando paradas a distancia caminable (combinaciones válidas). |
| for / forEach | MotorPuntaje.calcularRangos() y mejor() | Recorrer las alternativas; elegir la de mayor puntaje con una variable acumuladora. |
| while | Planificador (Dijkstra) | Procesar la cola de prioridad hasta llegar al destino. |
| while | SincronizadorHistorial.sincronizar() | Subir consultas pendientes mientras existan pendientes Y haya conexión de red. |

# 6. Algoritmos

## 6.1 Generación de alternativas (Planificador)

1. Valida los criterios (validar()) y que origen y destino estén a más de 200 m.
2. Agrega las alternativas directas permitidas: caminata (≤ 3,5 km), bicicleta (≤ 12 km) y taxi.
3. Para cada combinación de transporte público permitida (TransMilenio, SITP, ambos), busca el camino más rápido con **Dijkstra**.
4. Descarta las repetidas (misma «firma» de líneas). Si no queda ninguna, lanza RutaNoEncontradaException.

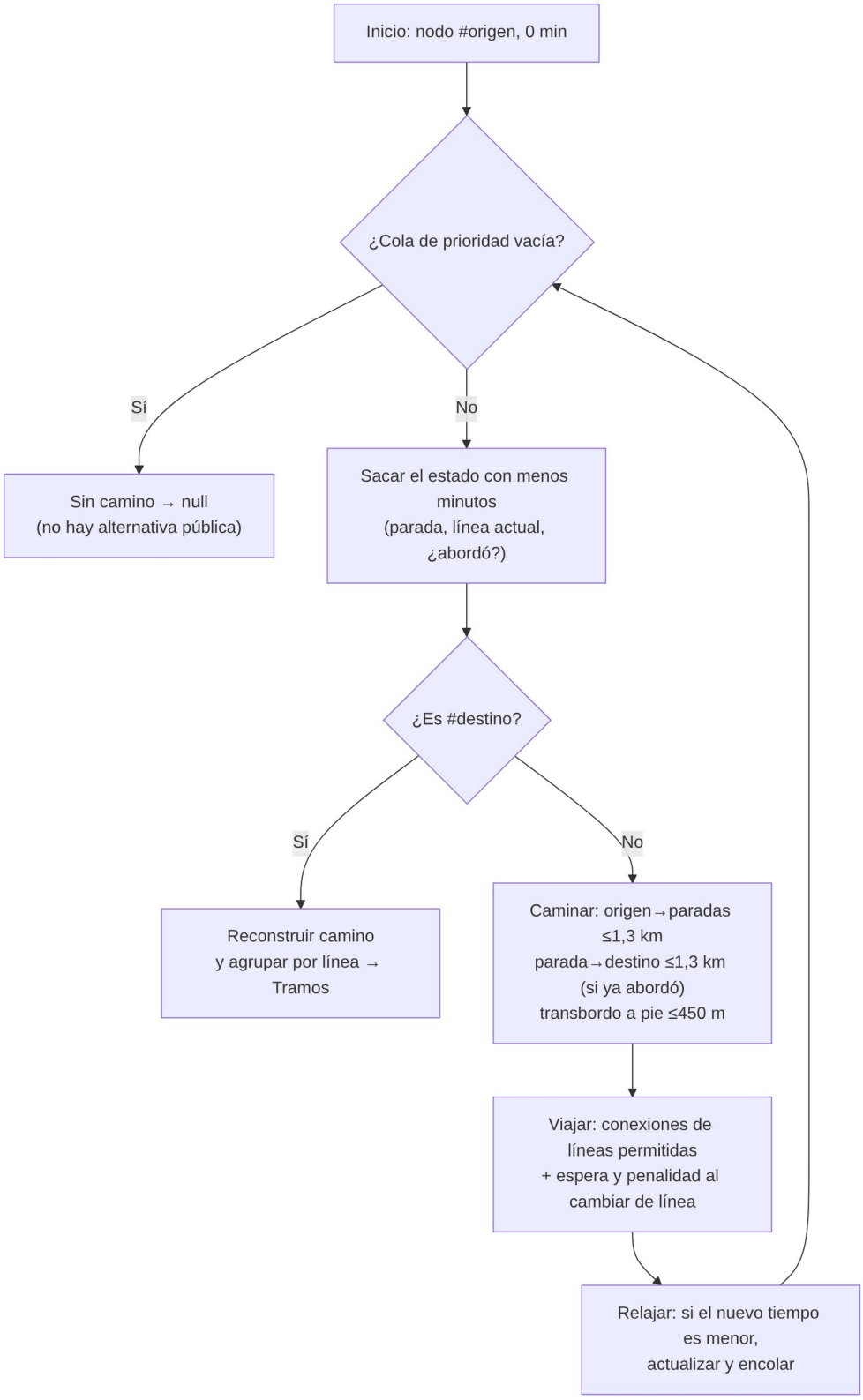



*Figura 3. Búsqueda del camino más rápido (Dijkstra)*

En Dijkstra, cada estado del grafo es (parada, línea actual, ¿ya abordó?):

- **Línea actual:** permite sumar la espera y una penalidad de 3 min al cambiar de línea.
- **¿Ya abordó?:** evita devolver un camino «solo a pie» como si fuera transporte público.

El camino encontrado se reconstruye y los pasos consecutivos de una misma línea se agrupan en un Tramo.

## 6.2 Motor de puntaje

Para cada criterio, la métrica de la alternativa se compara con la mejor de la búsqueda por proporción:

- tiempo, costo, transbordos y $CO_2$: (mejor + margen) / (valor + margen);
- comodidad: valor / mejor.

El margen evita que diferencias mínimas pesen demasiado: 10 min, $3.200, 1 transbordo y 500 g.

El puntaje es el promedio ponderado por los pesos del usuario, en escala 0–100. Las alternativas se ordenan de mayor a menor (en empate gana la más rápida) y la primera es la **recomendada**.

## 6.3 Ejemplo real calculado por la app

Trayecto Unicentro → Universidad Nacional, criterios por defecto (tiempo 5, costo 3, comodidad 3, transbordos 2, ambiente 1):

| # | Alternativa | Tiempo | Costo | Transbordos | $CO_2$ | Puntaje |
|---|---|---|---|---|---|---|
| 1 | En bicicleta | 48 min | $0 | 0 | 0 g | 77,9 |
| 2 | En taxi | 33 min | $19.800 | 0 | 1.964 g | 75,9 |
| 3 | TransMilenio + SITP | 65 min | $3.400 | 2 | 817 g | 48,6 |
| 4 | Solo bus SITP | 90 min | $3.400 | 1 | 1.440 g | 47,6 |

## 7. Persistencia

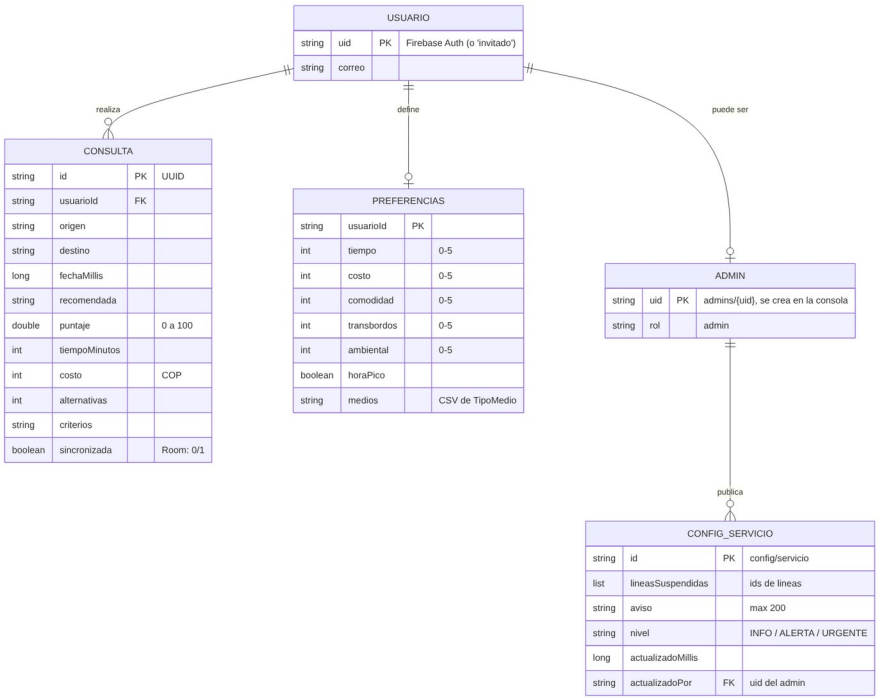



*Figura 4. Modelo de datos (Room y Firestore)*

- **Room** (rutasmart.db):
  - tabla consultas: historial, con la bandera sincronizada;
  - tabla preferencias: una fila por usuario.
  - Los DAO exponen Flow, así que la interfaz se actualiza sola.
- **Firestore**:
  - usuarios/{uid} guarda las preferencias (campo criterios);
  - usuarios/{uid}/consultas/{id} guarda el historial;
  - admins/{uid} marca a los administradores (solo se crea desde la consola de Firebase);
  - config/servicio guarda las líneas suspendidas y el aviso que publica el administrador; lo leen todos.

**Estrategia:** primero se escribe en Room, que funciona sin red. Después se sincroniza con un ciclo while mientras haya pendientes y conexión. Al iniciar sesión se descarga lo que haya en la nube.

# 8. Secuencia principal

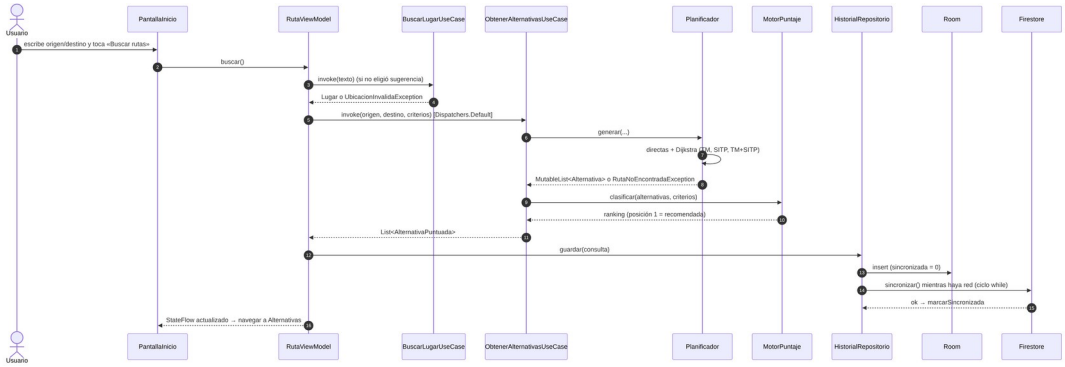



*Figura 5. Secuencia de una búsqueda*

## 9. Navegación

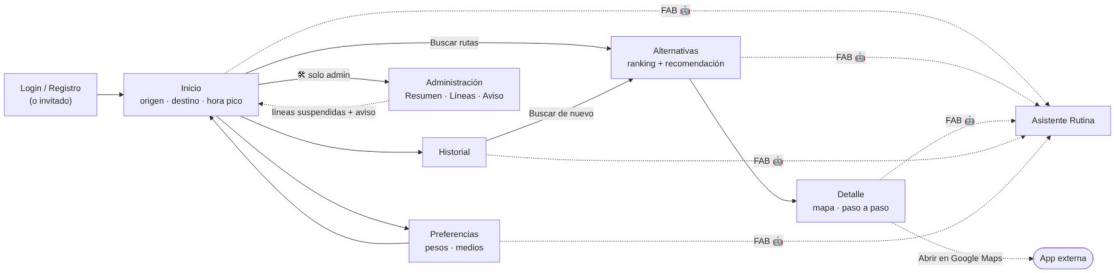



*Figura 6. Mapa de navegación*

La navegación usa **Navigation Compose** con pila: Inicio → Preferencias / Alternativas → Detalle. El asistente se abre con un **botón flotante** desde cualquier pantalla.

## 10. Manejo de excepciones

| Excepción | Cuándo se lanza | Mensaje al usuario |
|---|---|---|
| RutaNoEncontradaException | No hay combinación de paradas y medios entre origen y destino. | «No encontramos ninguna ruta… Prueba permitiendo más medios.» |
| UbicacionInvalidaException | Lugar vacío, no reconocido, ambiguo o igual al destino. | «No reconocemos «X» como origen. Elige un lugar de la lista…» |
| ServicioNoDisponibleException | Falla de red o de Firebase; clave o servicio caído. | «No hay conexión con el servicio. Verifica tu internet…» |
| CriteriosIncompletosException | Todos los pesos en 0 o ningún medio permitido. | «Define al menos un criterio…» / «Selecciona al menos un medio…» |

Todas heredan de RutaSmartException (clase sealed) con un mensajeUsuario.

- **ViewModels:** las capturan en bloques try-catch-finally. El finally siempre apaga el indicador de carga, el error se registra en la Bitacora y la interfaz muestra solo el mensaje comprensible.
- **Errores inesperados:** se capturan con un catch (e: Exception) general.
- **Asistente de IA:** sus errores se traducen con AiErrorMapper.

## 11. Seguridad contra valores nulos

| Técnica | Uso |
|---|---|
| Tipos nulables (?) | Lugar?, Linea?, Usuario?, PreferenciasEntity?, respuestas de Firestore y Gemini. |
| Llamada segura (?.) y Elvis (?:) | d.getString("origen") ?: return..., getLong(...)?.toInt() ?: 0, c.cause ?: e, it?.aDominio() ?: CriteriosUsuario(). |
| let | ui.mensaje?.let {{ ... }}, AiResponseGuard.clean(t)?.let {{ ... }}. |
| requireNotNull / require | ObtenerAlternativasUseCase valida origen y destino antes de calcular; RedTransporte valida paradas. |
| lateinit | Solo RutaSmartApp.contenedor, inicializado en onCreate() antes de cualquier uso. |

## 12. Depuración

- **Bitácora interna** (Bitacora): registra búsquedas, errores, sincronizaciones y fallas de la IA. En compilaciones de depuración se envía a **Logcat** con la etiqueta RutaSmart/<módulo>; en producción no se escribe nada en el log.
- **Depurador de Android Studio:** puntos de interrupción en Planificador.generar() y MotorPuntaje.clasificar() para inspeccionar alternativas y puntajes.
- **Layout Inspector:** verificación de la jerarquía de Compose y las etiquetas de accesibilidad.
- **Pruebas unitarias JUnit:** 80 pruebas que detectan errores de la lógica de comparación antes de ejecutar la app.
- **Monitoreo futuro:** integrar Firebase Crashlytics en la fase de pruebas con usuarios.

## 13. Implementación multiplataforma

El paquete domain/ (modelo, motor, casos de uso e IA) es Kotlin puro: no importa ninguna clase de Android. Por eso puede moverse a un módulo compartido de **Kotlin Multiplatform** (commonMain) y usarse desde iOS o desde una versión web (Kotlin/JS).

Las únicas piezas por plataforma serían la interfaz y los adaptadores de Room, Firebase y HTTP.

Para el cálculo de distancias (Geo) y la normalización de textos se usan funciones de java.*. En commonMain se reemplazarían por expect/actual.

## 14. Integración de inteligencia artificial

El asistente «Rutina» es **híbrido**:

1. El motor local (AsistenteLocal) calcula una respuesta exacta con el ranking.
2. Gemini recibe esa respuesta, los criterios y todas las alternativas puntuadas, y la redacta en lenguaje natural.

Si no hay clave, falla la red, se supera el límite o la respuesta es inválida, se muestra la respuesta local con el motivo. El detalle completo está en 07_Agente_IA/Agente_IA.

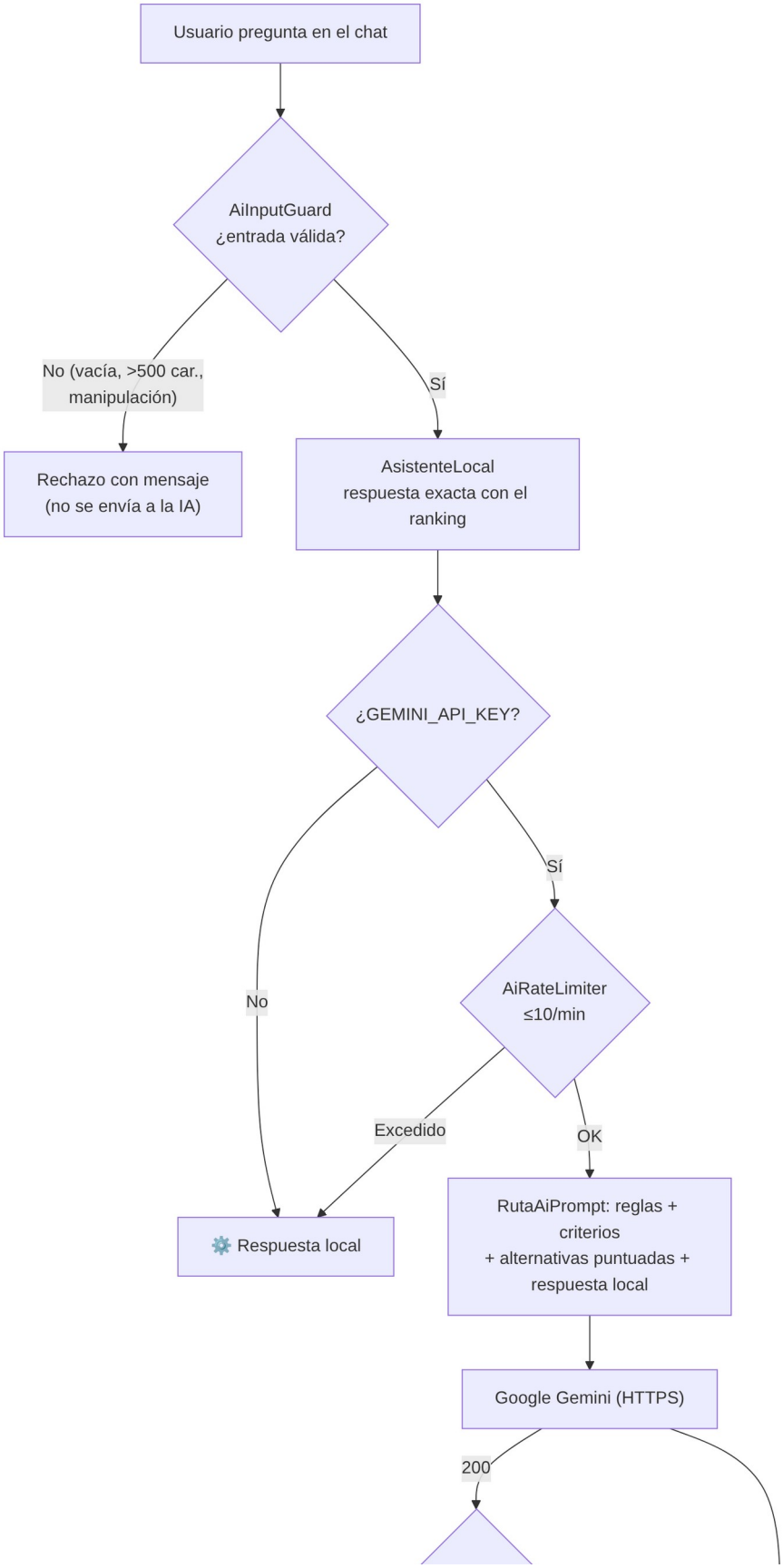

Usuario pregunta en el chat
AiInputGuard
¿entrada válida?
No (vacía, >500 car., manipulación)
Sí
Rechazo con mensaje
(no se envía a la IA)
AsistenteLocal
respuesta exacta con el ranking
¿GEMINI_API_KEY?
No
Sí
AiRateLimiter
≤10/min
Excedido
OK
⚙️ Respuesta local
RutaAiPrompt: reglas + criterios
+ alternativas puntuadas + respuesta local
Google Gemini (HTTPS)
200

*Figura 7. Flujo del agente de IA*

## 15. Estructura del código

| Paquete | Contenido |
|---|---|
| ui/ | ViewModels.kt (Sesion, Ruta, Asistente, Admin), navegacion/Navegacion.kt, pantallas/ (8 pantallas + componentes), tema/Tema.kt |
| domain/model | Parada, Lugar, Linea, Conexion, MedioDeTransporte y sus 5 clases, Ruta, Tramo, Geo, Alternativa, CriteriosUsuario, ConsultaHistorial, excepciones |
| domain/motor | RedTransporte, Planificador, MotorPuntaje, Bitacora |
| domain/ casosdeuso | BuscarLugarUseCase, ObtenerAlternativasUseCase, RecomendarRutaUseCase, ValidadorCuenta |
| domain/admin | EstadoServicio, NivelAviso, AdminValidador, RegistroUso, ResumenUso, Estadisticas |
| domain/ia | AiCore (guardas, límites, errores), AsistenteLocal, RutaAiPrompt |
| data/local | Room: entidades, DAO y base de datos |
| data/remoto | FirebaseRemoto (Auth, Firestore y funciones de administración), GeminiClient |
| data/ repositorio | Historial, Preferencias, Asistente, Servicio (administración) y EstadoRed |
| data/red | RedBogota: 50 paradas, 10 líneas y 19 lugares |